



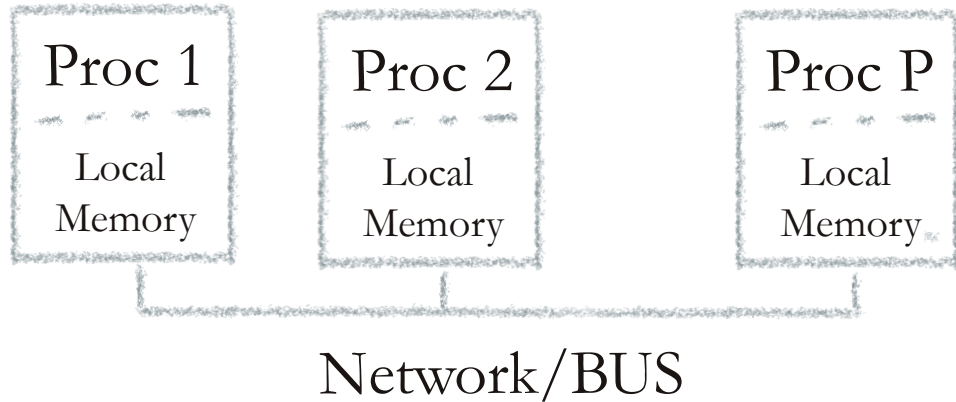
OpenMP

Dr. Torsten Bosse
Advanced Computing, FSU Jena

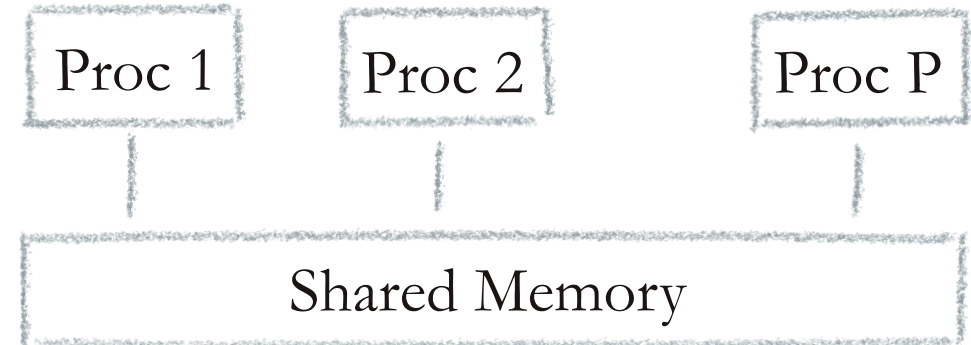
OpenMP

- OpenMP Programming Model
- Fork/Join
- Variable Scoping
- Clauses
- Parallel For/...
- Timing
- Examples
- False Sharing
- Race Condition
- Synchronization (Critical, Atomic,...)
- Scheduling
- OpenMP Tasks

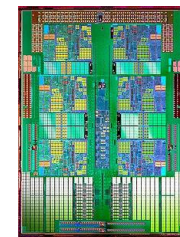
Distributed System vs. Shared Memory System



- Processors have local/private memory
- Communication via Network Messages
- Data-Movement expensive
- Easily scalable/extendable



- Processors share memory
- Communication via Memory
- Data-Movement “Cheap”
- Only limited scalable



Open Multi-Processing (OpenMP)

OpenMP is an application programming interface that supports

- multi-platform shared-memory multiprocessing programming in
- different languages like C, C++, and Fortran, on
- many platforms, instruction-set architectures and
- various operating systems, including Linux, macOS, and Windows.

OpenMP consists of a set of

- *compiler directives,*
- *library routines, and*
- *environment variables*

that influence run-time behaviour.

<https://www.openmp.org>

“Hello World” - Example

```
#include <stdio.h>

int main(){

    printf("Hello World\n");

    return 0;
}
```

➤ `gcc -o HelloWorld HelloWorld.c`

➤ `./HelloWorld`

Standard C/C++ Example

Compilation with GCC Compiler

Execution of compiled file

OpenMP “Hello World” Example Reloaded

```
#include <stdio.h>
#include <omp.h>

int main(){
#pragma omp parallel
{
    printf("Hello World\n");
}
return 0;
}
```

➤ `gcc -o OpenMP-HelloWorld -fopenmp OpenMP-HelloWorld.c`

➤ `./OpenMP-HelloWorld`

OpenMP C/C++ Example

Compilation as OpenMP Code

Execution of compiled file

OpenMP “Hello World” Example Reloaded

```
#include <stdio.h>
#include <omp.h>

int main(){
#pragma omp parallel
{
    printf("Hello World\n");
}
return 0;
}
```

```
➤ gcc -o OpenMP-HelloWorld -fopenmp OpenMP-HelloWorld.c
```

```
➤ ./OpenMP-HelloWorld
```

```
➤ ./OpenMP-HelloWorld
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
...
```

[OpenMP API Specification 5.2](#)

Programming Model

```
#pragma omp directive [clause list]
```

General structure for OpenMP compiler pragmas.

```
#pragma omp parallel [clause list]  
/*Structure Block*/
```

Threads created by parallel directive execute the code in the structured block in parallel according to the conditions specified in the clause list.

Clause Type	Code	Explanation
Conditional Parallelisation	if (scalar expr)	Determines whether the parallel construct results in creation of thread.
Degree of Concurrency	num_threads (integer expr)	Specify the number of threads to be create
Data Handling	private (variable list)	Specify set of variables to be local for threads
	shared (variable list)	Specify set of variables to be shared by threads
...		...

OpenMP Translated Code

OMP-Pragma tells compiler how to “distribute” the code (parts)!!!

Original Code

```
#include <omp.h>
int a, b;
void main(){
// Serial Segment
#pragma omp parallel num_threads (8) private (a) shared (b)
{
    // Parallel Segment
}
// Rest of Serial Segment
}
```

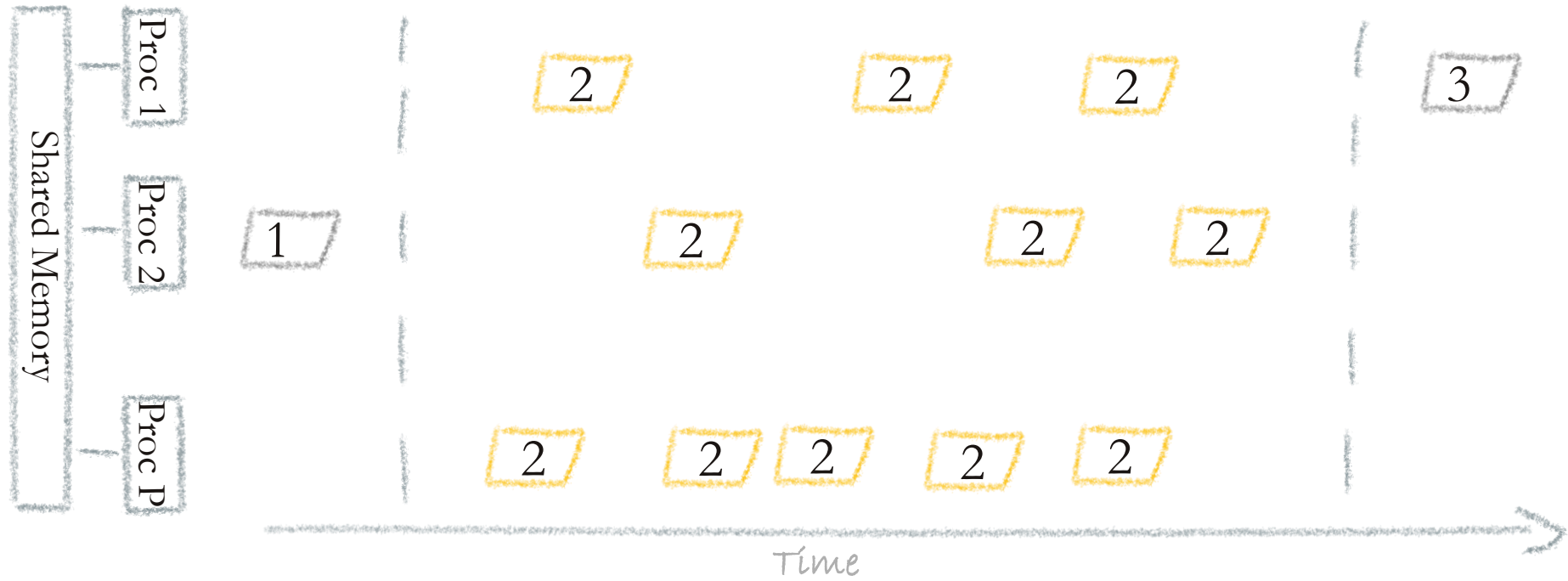
Translated Code

```
#include <pthread.h>

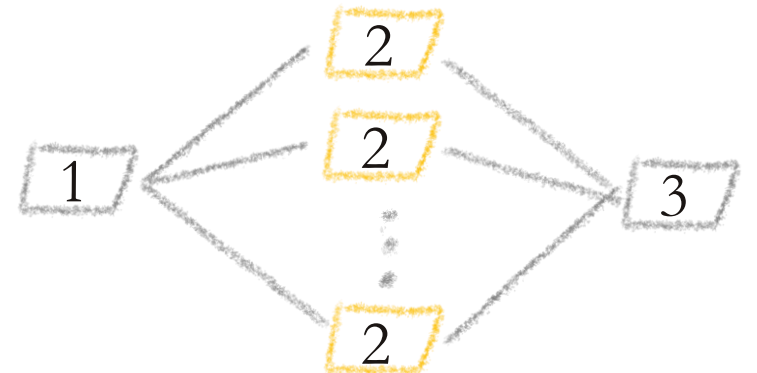
int a,b;
void main () {
    //Serial Segment
    for(int i=0; i< 8; i++)
        pt_thread_create (... , internal_th_fn_name, ...);
    for(int i=0; i< 8; i++)
        pt_thread_join (...);
    //Rest of serial Segment
}

void * internal_th_fn_name (void *packaged_argument){
    int a;
    //Parallel Segment
}
```

Threads to Processors



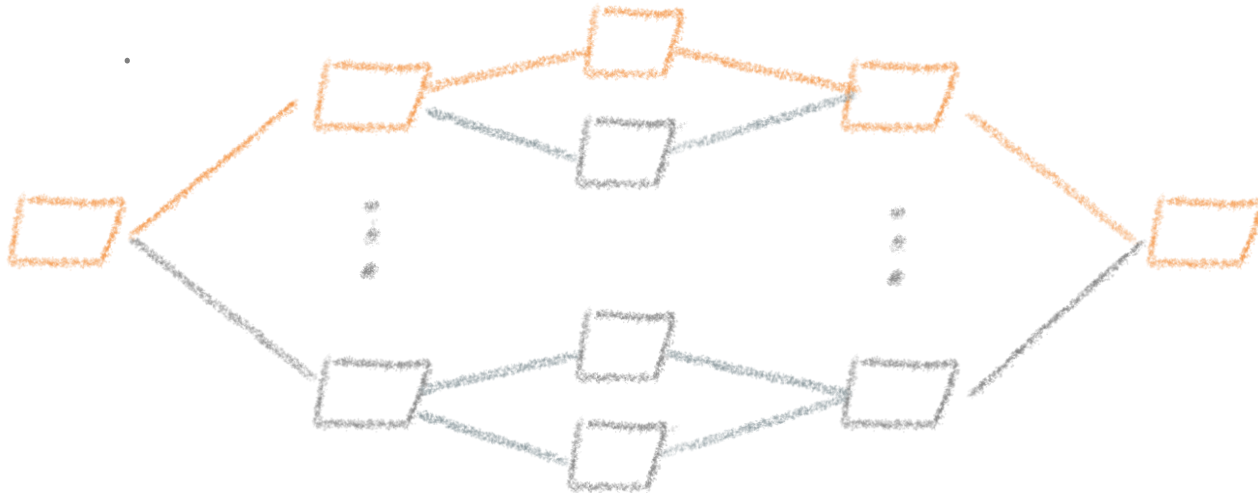
```
//Code Block 1  
#pragma omp parallel  
{  
    //Code Block 2  
}  
//Code Block 3
```



Nested Fork/Join

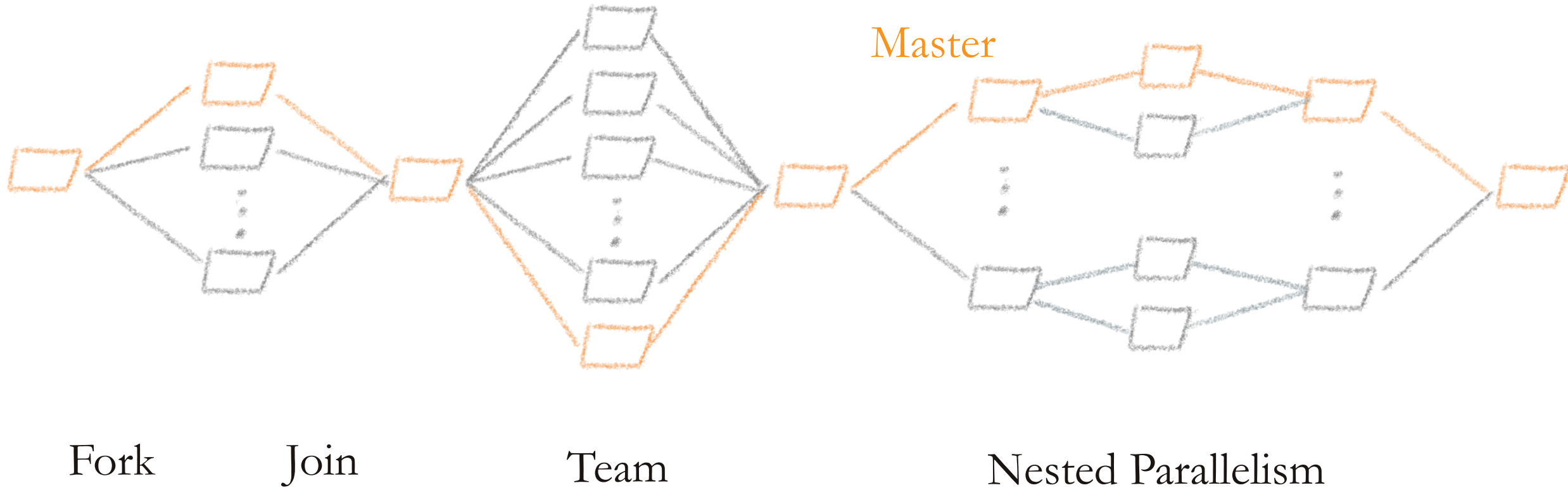


```
#pragma omp parallel
{
    //Code Block
}
```



```
#pragma omp parallel
{
    //Code Block
    #pragma omp parallel
    {
        //Code Block
    }
    //Code Block
}
```

Naming Convention



OpenMP “Hello World” Example Reloaded

```
#include <stdio.h>
#include <omp.h>

int main(){
#pragma omp parallel
{
    printf("Hello World... from thread = %d\n",
          omp_get_thread_num());
}
return 0;
}
```

➤ `gcc -o OpenMP-Thread_Num -fopenmp OpenMP-Thread_Num.c`

➤ `./OpenMP-Thread_Num`

OpenMP C/C++ Example

Compilation as OpenMP Code

Execution of compiled file

OpenMP “Hello World” Example Reloaded

```
#include <stdio.h>
#include <omp.h>

int main(){
#pragma omp parallel
{
    printf("Hello World... from thread = %d\n",
        omp_get_thread_num());
}
return 0;
}
```

➤ `gcc -o OpenMP-Thread_Num -fopenmp OpenMP-Thread_Num.c`

➤ `./OpenMP-Thread_Num`

➤ `./OpenMP-Thread_Num`

```
Hello World... from thread = 21
Hello World... from thread = 13
Hello World... from thread = 18
Hello World... from thread = 1
Hello World... from thread = 5
Hello World... from thread = 22
Hello World... from thread = 6
Hello World... from thread = 8
```

...

[OpenMP API Specification 5.2](#)

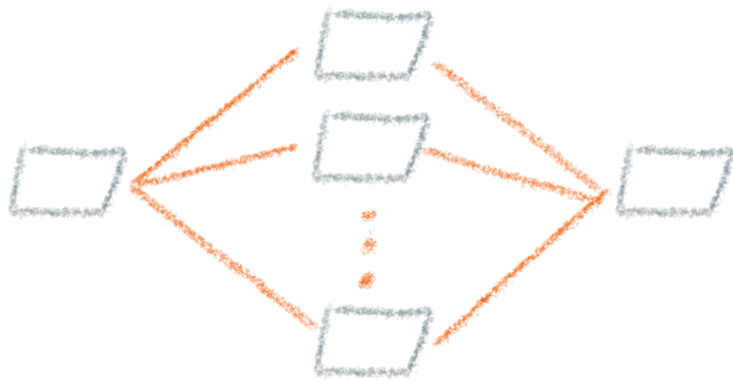
OpenMP “Hello World” Example Reloaded

```
#include <stdio.h>
#include <omp.h>

int main(){
#pragma omp parallel
{
    printf("Hello World... from thread = %d\n",
           omp_get_thread_num());
}
return 0;
}
```

```
> ./OpenMP-Thread_Num
Hello World... from thread = 21
Hello World... from thread = 13
Hello World... from thread = 18
Hello World... from thread = 1
Hello World... from thread = 5
Hello World... from thread = 22
Hello World... from thread = 6
Hello World... from thread = 8
```

...



Set number of required threads

```
> export OMP_NUM_THREADS=5
```

Thread Number Methods

```
void omp_set_num_threads(int num_threads);
```

Set the number of threads to be used for subsequent parallel regions that do not specify a **num_threads** clause.

```
int omp_get_num_threads(void);
```

Return the number of threads in the current team.

```
int omp_get_thread_num(void);
```

Return the thread number, within the current team, of calling thread.

```
int omp_get_max_threads(void);
```

Return an upper bound on the number of threads available in a new team for a subsequent parallel construct without **num_threads** clause.



Thread Number Example

```
#include <stdio.h>
#include <omp.h>

void report(int CP){
    printf("CP %d (%d, %d, %d)\n", CP, omp_get_thread_num(),
        omp_get_num_threads(), omp_get_max_threads());
}

int main(){
    report(0);
    omp_set_num_threads(3);
    #pragma omp parallel
    {
        report(1);
    }
    report(2);
    omp_set_num_threads(5);
    #pragma omp parallel
    {
        report(3);
    }
    return 0;
}
```



```
> gcc -o OpenMP-Thread_Num_Example -fopenmp
OpenMP-Thread_Num_Example.c
```

```
> export OMP_NUM_THREADS=5
```

```
> ./OpenMP-Thread_Num_Example
```

```
*****
😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊
*****
...
```

Thread Number Example II

```
//OpenMP-Num_Threads.c
#include<omp.h>
#include<stdio.h>
#include<stdlib.h>

void main(){
    #pragma omp parallel num_threads(7)
    {
        int id = omp_get_thread_num();
        printf("Thread Id = %d \n",id);
        if(id == 0){
            int nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n",nthreads);
        }
    }
}
```

```
#pragma omp parallel num_threads(7)
```

The number of threads to be used for a subsequent parallel region can also be specified by pragma directive



Timing

```
double omp_get_wtime(void);
```

Return the elapsed wall clock time in seconds.

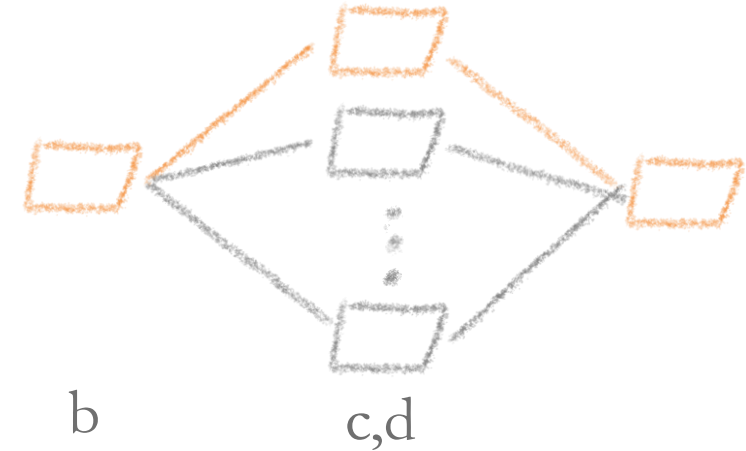
```
#include <stdio.h>
#include <omp.h>
int main(){
    double start = omp_get_wtime();
    ... work to be timed ...
    double end = omp_get_wtime();
    printf(" CP1 - Work took %f seconds\n", end - start);
    #pragma omp parallel
    {
        start = omp_get_wtime();
        ... work to be timed ...
        end = omp_get_wtime();
        printf(" CP2 - Work took %f seconds\n", end - start);
    }
    end = omp_get_wtime();
    printf(" CP3 - Work took %f seconds\n", end - start);
    return 0;
}
```

➤ `gcc -o OpenMP-Timing_Parallel -fopenmp OpenMP-Timing_Parallel.c; ./OpenMP-Timing_Parallel`

😊😊😊😊 FIND OUT AND TRY TO EXPLAIN THE RESULT 😊😊😊😊

Scoping Experiment I

```
#include <stdio.h>
#include <omp.h>
static int a=0;
int main(){
    int b=0;
    omp_set_num_threads(10);
    #pragma omp parallel
    {
        int c=0;
        static int d=0;
        a++;
        b++;
        c++;
        d++;
        printf("Thread, (a,b,c,d) = %d, (%d,%d,%d,%d)\n",
               omp_get_thread_num(), a, b, c, d);
    }
    printf("Thread, (a,b) = %d, (%d,%d)\n",
           omp_get_thread_num(), a, b);
    return 0;
}
```



➤ `gcc -o OpenMP-Scoping -fopenmp OpenMP-Scoping.c; ./OpenMP-Scoping`

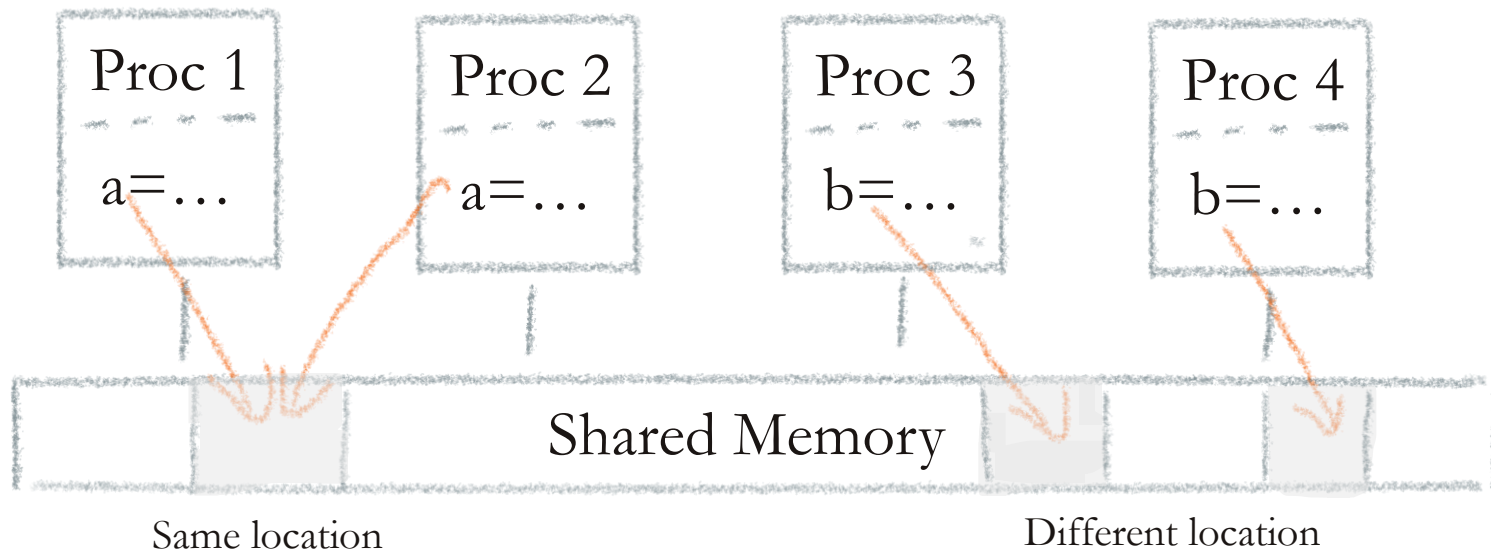
```
*****
*
😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊
*****
*
...
```

OpenMP Scoping

OpenMP declares by default certain variables as either “shared” or “private” .

Shared variables refer to *same memory locations* (in shared memory) if accessed by two or more processors.

Private variables refer to *different memory locations* (in shared memory), which are private to each of the processors.



Scoping Default Behavior

- Static Variables are (usually) shared.
- Variable declared outside the parallel region are (usually) shared
- Variables which are declared locally within a parallel region are private.
- loop iteration variables are private by default



```
#include <stdio.h>
#include <omp.h>
static int a=0;
int main(){
    int b=0;
    omp_set_num_threads(10);
    #pragma omp parallel
    {
        int c=0;
        static int d=0;
        a++;
        b++;
        c++;
        d++;
        printf("Thread, (a,b,c,d) = %d, (%d,%d,%d,%d)\n",
               omp_get_thread_num(), a, b, c, d);
    }
    printf("Thread, (a,b) = %d, (%d,%d)\n",
           omp_get_thread_num(), a, b);
    return 0;
}
```

Scoping Clauses Programming Model

```
#pragma omp parallel [clause list]
/*Structure Block*/
```

OpenMP allows to specify, which variables in the parallel code block are private or shared by certain OMP clauses.

Clause Type	Example	Explanation
private(list)	<code>#pragma omp parallel private(a,b)</code>	Specify the variables in the list to be private
shared(list)	<code>#pragma omp parallel shared(a,b)</code>	Specify the variables in the list to be shared
default(none)	<code>#pragma omp parallel default(none) private(a) shared(b)</code>	Specify that all variables need to be specified as either private or shared
default(shared)	<code>#pragma omp parallel default(shared)</code>	Specify that by default all variables are shared
...		...

Privat Example

```
#include <stdio.h>
#include <omp.h>
int main() {
    int counter = 0;
    //Activate private option to see effects
    #pragma omp parallel num_threads(10) //private(counter)
    {
        int tid = omp_get_thread_num();
        counter += tid;
        printf("Thread %d: counter = %d\n", tid, counter);
    }
    printf("Outside parallel region - counter: %d\n", counter);
    return 0;
}
```

➤ *gcc -o OpenMP-Private -fopenmp OpenMP-Private.c; ./OpenMP-Private*

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

...

Scoping Examples

```
#pragma omp parallel for shared(n, a)
for (int i = 0; i < n; i++)
{
    int b = a + i;
    ...
}
```

```
#pragma omp parallel for shared(n, a) private(b)
for (int i = 0; i < n; i++)
{
    b = a + i;
    ...
}
```

```
int a, b, c, n;
...

#pragma omp parallel for default(shared)
for (int i = 0; i < n; i++)
{
    // using a, b, c
}
```

```
int n = 10;
std::vector<int> vector(n);
int a = 10;
#pragma omp parallel for default(none) shared(n, vector, a)
for (int i = 0; i < n; i++)
{
    vector[i] = i * a;
}
```

FirstPrivate Experiment

```
#include <stdio.h>
#include <omp.h>
int main() {
    int var = 0;
    #pragma omp parallel for private(var) num_threads(2)
    for (int i = 0; i < 10; i++) {
        var += 1;
        printf("Thread %d: var = %d\n", omp_get_thread_num(), var);
    }
    printf("Outside parallel region - var: %d\n\n", var);
    var = 0;
    #pragma omp parallel for private(var) num_threads(2)    // Change "private" to "firstprivate" to see effects

    for (int i = 0; i < 10; i++) {
        var += 1;
        printf("Thread %d: var = %d\n", omp_get_thread_num(), var);
    }
    printf("Outside parallel region - var: %d\n", var);
    return 0;
}
```

LastPrivate Experiment

```
#include <stdio.h>
#include <omp.h>
int main() {
    int var = 0;
    #pragma omp parallel for private(var) num_threads(2)
    for (int i = 0; i < 10; i++) {
        var += 1;
        printf("Thread %d: var = %d\n", omp_get_thread_num(), var);
    }
    printf("Outside parallel region - var: %d\n\n", var);
    var = 0;
    #pragma omp parallel for private(var) num_threads(2)    // Change "private" to "lastprivate" to see effects
    for (int i = 0; i < 10; i++) {
        var += 1;
        printf("Thread %d: var = %d\n", omp_get_thread_num(), var);
    }
    printf("Outside parallel region - var: %d\n", var);
    return 0;
}
```

FirstPrivate/LastPrivate Clauses

Private variables are only local to a parallel thread block. How to initialize these private variables and return their values back after the parallel region?

Clause	Example	Explanation
firstprivate(list)	<code>#pragma omp parallel firstprivate(a)</code>	Specifies that each thread should have its own instance of a variable, and that the variable should be initialized with the value of the variable, because it exists before the parallel construct.
lastprivate(list)	<code>#pragma omp parallel lastprivate(a)</code>	Specifies that the enclosing context's version of the variable is set equal to the private version of whichever thread executes the final iteration (for-loop construct) or last section (#pragma sections)
...		...

Scoping Example I

```
#include <stdio.h>
#include <omp.h>

int main (){
    int i = -1;
    #pragma omp parallel private(i)
    {
        i = 0;
        printf("thread %d: i = %d\n", omp_get_thread_num(), i);
        i = omp_get_thread_num();
    }
    printf("i = %d\n", i);
    return 0;
}
```

➤ `gcc -o OpenMP-Scoping_Example_I -fopenmp OpenMP-Scoping_Example_I.c`

➤ `./OpenMP-Scoping_Example_I`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Scoping Example II

```
#include <stdio.h>
#include <omp.h>

int main (){
    int i = 10;
    #pragma omp parallel firstprivate(i)
    {
        printf("thread %d: i = %d\n", omp_get_thread_num(), i);
        i = 1000 + omp_get_thread_num();
    }
    printf("i = %d\n", i);
    return 0;
}
```

➤ `gcc -o OpenMP-Scoping_Example_II -fopenmp OpenMP-Scoping_Example_II.c`

➤ `./OpenMP-Scoping_Example_II`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Scoping Example III

```
#include <stdio.h>
#include <omp.h>

int main ()
{
    int i = -1;
    int j = -1;
    #pragma omp parallel for firstprivate(i) lastprivate(j)
        for (int k=0;k<2;k++)
        {
            j = k;
            printf("thread %d: i = %d, j = %d\n", k, i, j);
        }
    printf("i = %d, j = %d\n", i, j);
    return 0;
}
```

➤ `gcc -o OpenMP-Scoping_Example_III -fopenmp OpenMP-Scoping_Example_III.c`

➤ `./OpenMP-Scoping_Example_III`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Clauses

Besides specifying private and shared variables, the clauses of an OpenMP pragma can be used to control the behaviour of the threads.

Pragma	Example	Explanation
parallel	<code>#pragma omp parallel</code>	Spawn and run parallel threads
single	<code>#pragma omp single</code>	Run block only on a single thread
master	<code>#pragma omp master</code>	Run block only on a master thread
if(cond)	<code>#pragma omp parallel if(cond)</code>	Run block in parallel if cond is true
for	<code>#pragma omp parallel for</code>	Run loop of for-loop in parallel
for reduction	<code>#pragma omp parallel for reduction(op:List)</code>	Run loop of for-loop in parallel
...		...

Single/If Clause

```
#include <stdio.h>
#include <omp.h>

void test(int val){
    #pragma omp parallel if (val)
    if (omp_in_parallel()){
        #pragma omp single
        printf("val = %d, parallelized with %d threads\n",
            val, omp_get_num_threads());
    }
    else{
        printf_s("val = %d, serialized\n", val);
    }
}

void main( ){
    omp_set_num_threads(2);
    test(0);
    test(2);
}
```

➤ `gcc -o OpenMP-Clause_If -fopenmp OpenMP-Clause_If.c; ./OpenMP-Clause_If`

☺ FIND OUT AND EXPLAIN RESULT ☺



Hint: Try the same code but replace single by master

Parallel For-Directive

```
#pragma omp parallel
{
    #pragma omp for
    for(int i = 0; i < n; i++){
        //Code Block
    }
}
```

```
#pragma omp parallel for
for(int i = 0; i < n; ++i){
    //Code Block
}
```

The “parallel for”-directive instructs the compiler to distribute loop iterations within the team of threads that encounters this work-sharing construct.



Parallel For - Example

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main (){
    int n = 10;
    int* a = (int *)malloc(n*sizeof(int));

    #pragma omp parallel for shared(a)
        for(int i = 0; i < n; i++)
            a[i] = i;

    #pragma omp single
    for(int i = 0; i < n; i++)
        printf("%d,",a[i]);

    return 0;
}
```

What is happening if the loop iterations are not independent of each other but dependent, e.g. $a[i+1]=a[i]$?

What happens if “shared(a)” is not specified?

Try to formulate a parallel algorithm that sums the entries of a vector of length n ...!

Make runtime experiments!

Reduction Clauses

- Reduction clauses are data-sharing attribute clauses that can be used to perform some forms of recurrence calculations in parallel.
- Reduction clauses include reduction scoping clauses and reduction participating clauses.
- Define the region in which a reduction is computed.
- Reduction participating clauses define the participants in the reduction.
- Reduction clauses specify a reduction-identifier and one or more list items



```
#pragma omp parallel for reduction(op:list)
```

op	Initializer	Combiner
+	priv = 0	out += in
-	priv = 0	out -= in
*	priv = 1	out *= in
&	priv = ~ 0	out &= in
	priv = 0	out = in
^	priv = 0	out ^= in
&&	priv = 1	out = in && out
	priv = 0	out = in out
max	priv = <i>Least representable number in the reduction list item type</i>	out = in > out ? in : out
min	priv = <i>Largest representable number in the reduction list item type</i>	out = in < out ? in : out

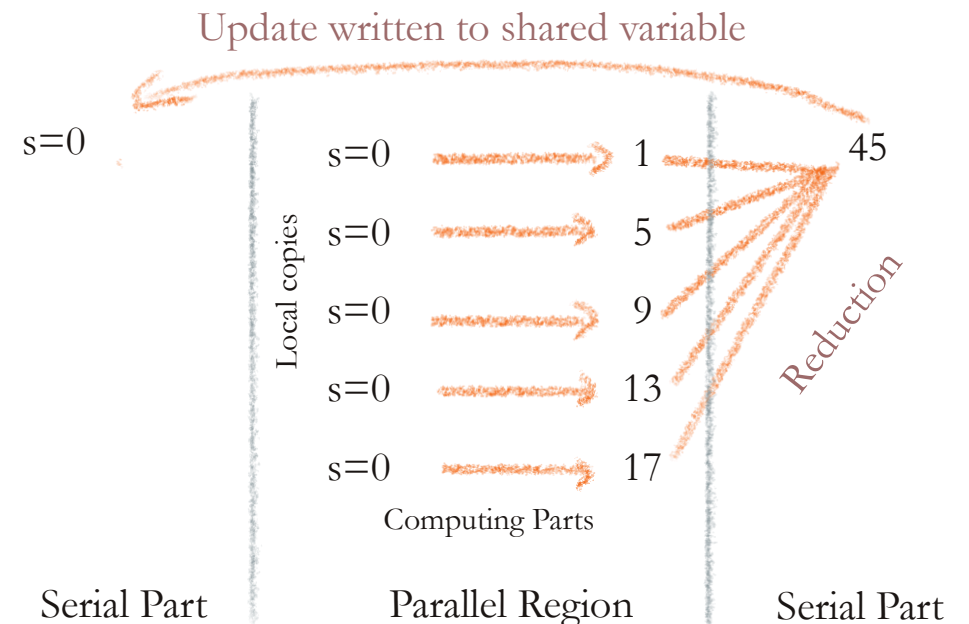
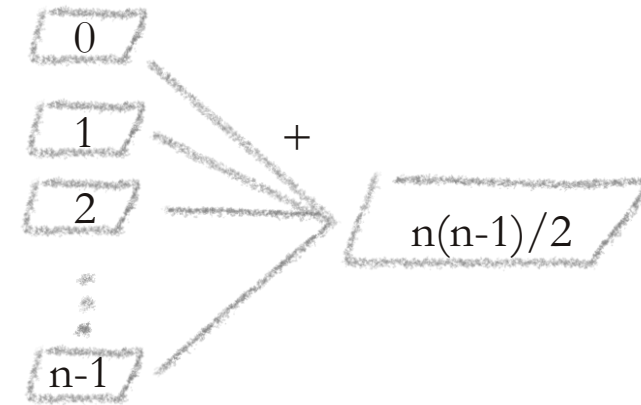
Reduction Example

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

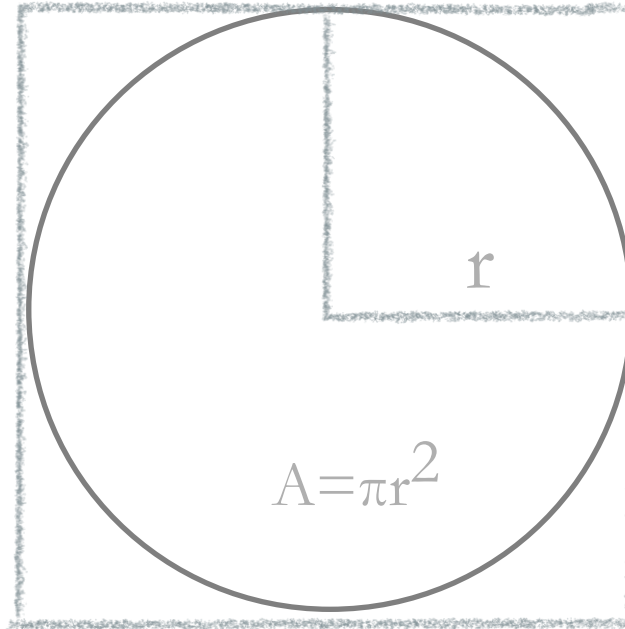
void main (){
    long int n = 10;
    int s = 0;
    int* a = (int *)malloc(n*sizeof(int));
    #pragma omp parallel for shared(a)
    for(long int i = 0; i < n; i++){
        a[i] = i;
    }

    #pragma omp parallel for reduction(+:s)
    for(long int i = 0; i < n; i++){
        s += a[i];
    }

    printf("s is %d \n", s);
}
```



PI - Example



3.1415926535...

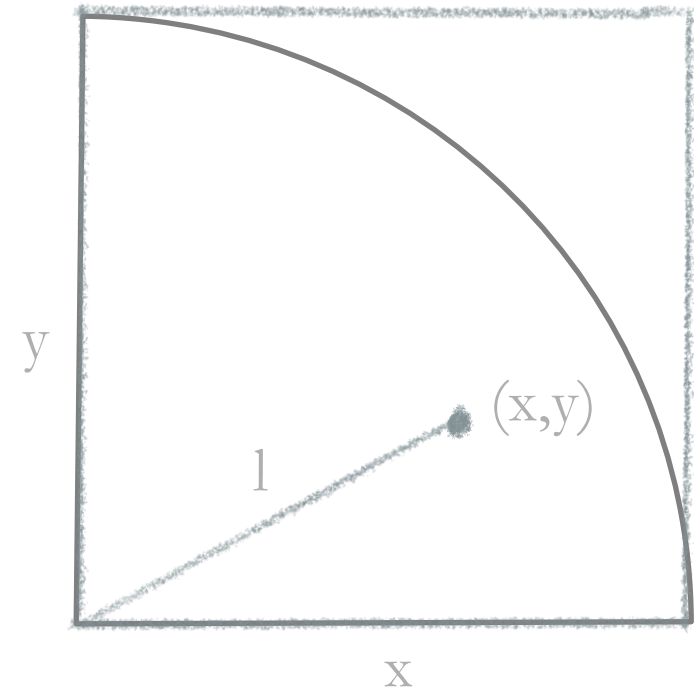
Task: Compute an approximation of the number π .

PI - Example via Monte Carlo

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    long int N = 100000000;
    long int I = 0;
    double x, y, l, pi;
    srand( time(NULL) );
    for(long int i = 0; i < N; i++) {
        x = (double)rand() / RAND_MAX;
        y = (double)rand() / RAND_MAX;
        l = sqrt(x * x + y * y);
        if( l <= 1 ) I++;
    }
    pi = (double) I / N * 4;
    printf("Approximated PI = %g\n", pi);
    return 0;
}
```

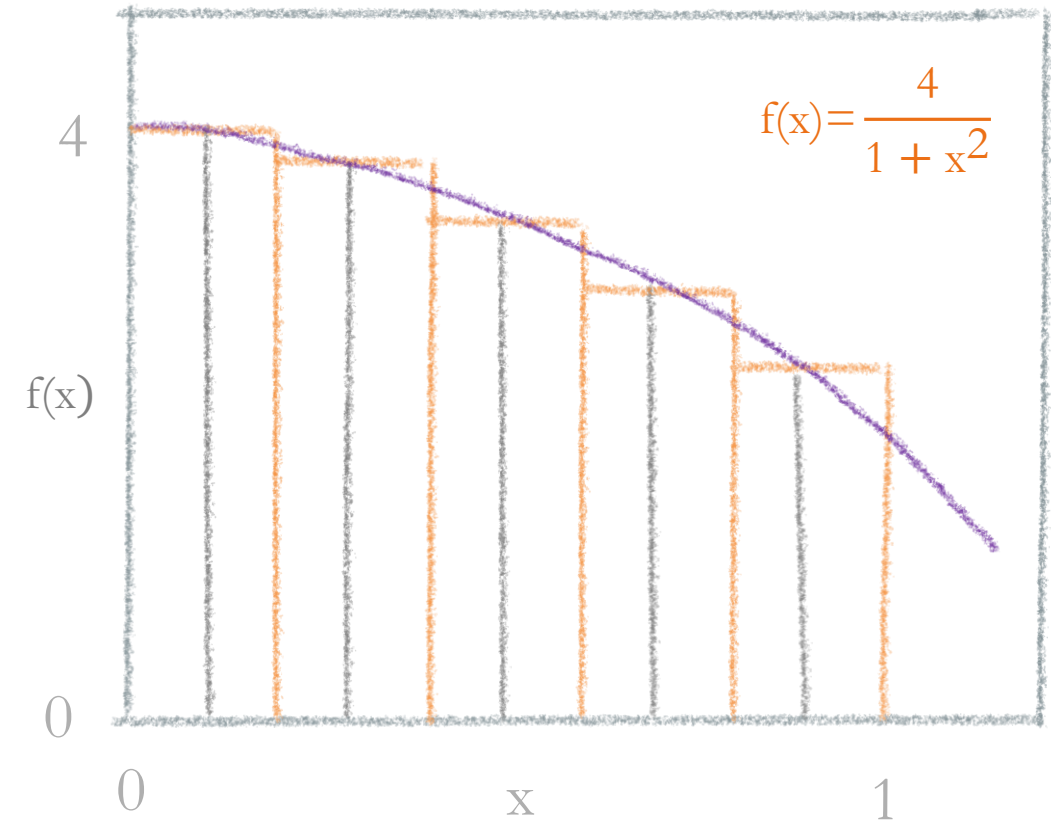
Hint: Use “-lm” when compiling to include math.h



$$\text{Approximation of } \pi \approx \frac{4 \cdot I}{N}$$

PI - Example via Integral

```
#include <stdio.h>
int main ()
{
    long int N = 100000000;
    double t, pi, sum = 0.0;
    double h= 1.0/(double) N;
    for(long int i=0;i< N; i++){
        t = (i+0.5)*h;
        sum = sum + 4.0/(1.0+t*t);
    }
    pi = h * sum;
    printf("Approximated PI = %g\n", pi);
    return 0;
}
```



$$\text{Approximation of } \pi = \int_0^1 \frac{4}{1+x^2} dx \approx \sum_{i=0}^{N-1} \left[\frac{4h}{1+((i+0.5)h)^2} \right]$$

PI - Example via Integral (Simple)

```
#include <stdio.h>
#include <omp.h>
#define MAX_THREADS 4
static long num_steps = 100000000;
double step;
int main (){
    int i,j;
    double pi, full_sum = 0.0;
    double start_time, run_time;
    double sum[MAX_THREADS];
    step = 1.0/(double) num_steps;

    for (j=1;j<=MAX_THREADS ;j++) {
        omp_set_num_threads(j);
        full_sum=0.0;
        start_time = omp_get_wtime();
        #pragma omp parallel
        {
```

...

```
...
        int i;
        int id = omp_get_thread_num();
        int numthreads = omp_get_num_threads();
        double x;
        sum[id] = 0.0;
        if (id == 0) printf(" num_threads = %d",numthreads);
        for (i=id;i< num_steps; i+=numthreads){
            x = (i+0.5)*step;
            sum[id] = sum[id] + 4.0/(1.0+x*x);
        }
    }
    for(full_sum = 0.0, i=0;i<j;i++){
        full_sum += sum[i];
    }
    pi = step * full_sum;
    run_time = omp_get_wtime() - start_time;
    printf("\n pi is %f in %f seconds %d thrds \n",pi,run_time,j);
}
```

PI - Example via Integral (Final)

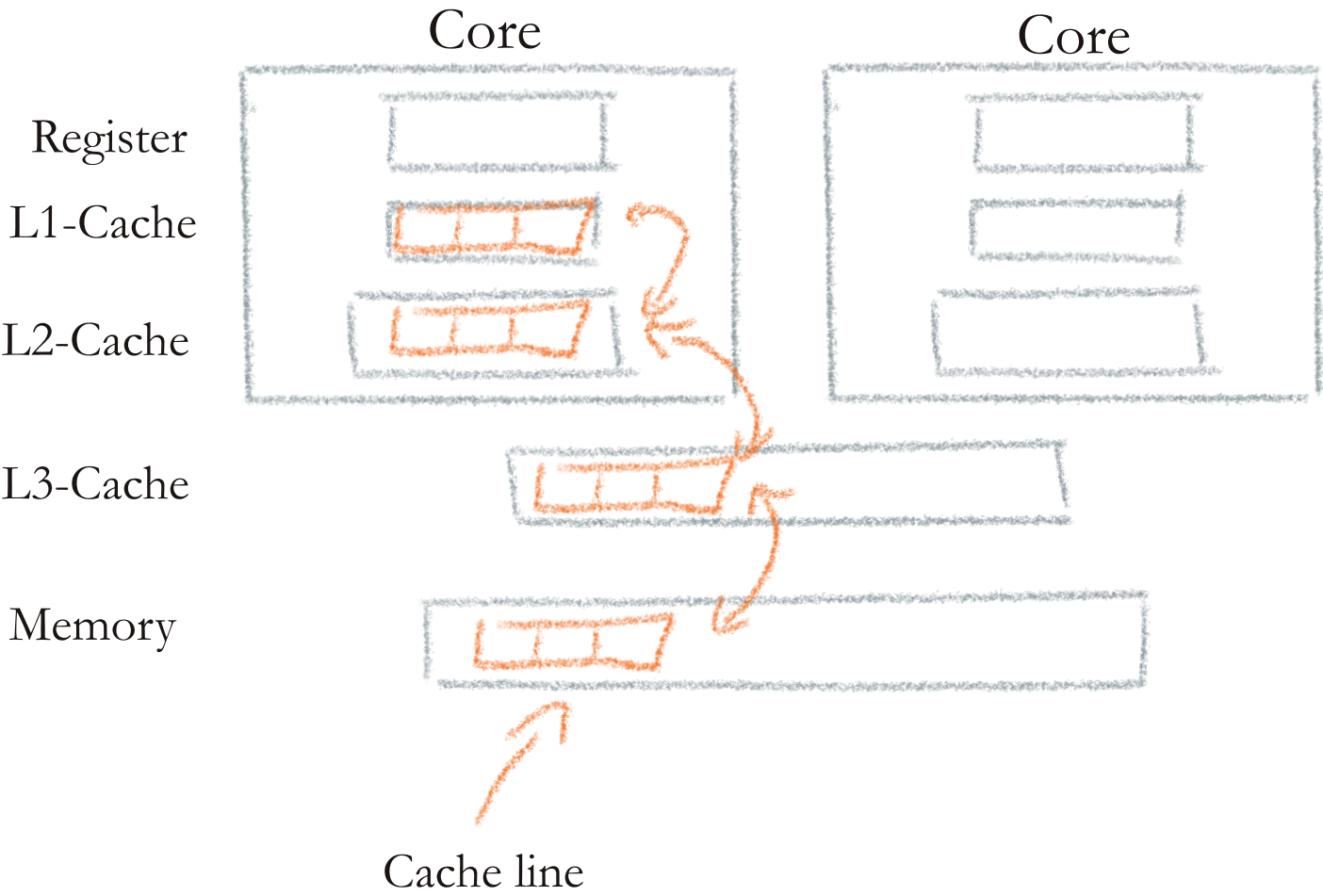
```
#include <omp.h>
#include <stdio.h>
#define MAX_THREADS 4

static long num_steps = 100000000;
double step;
int main() {
    int i, j;
    double pi, full_sum = 0.0;
    double start_time, run_time;

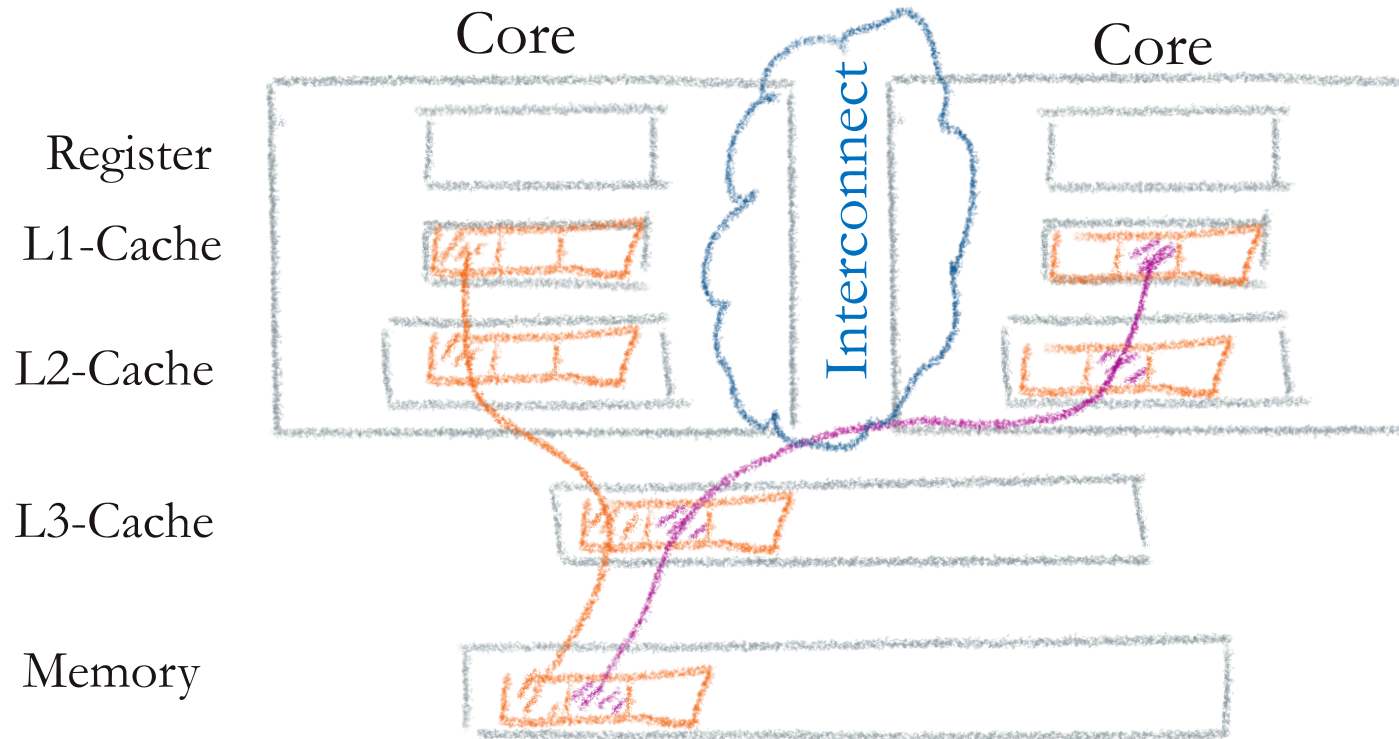
    step = 1.0 / (double)num_steps;
    for (j = 1; j <= MAX_THREADS; j++) {
        omp_set_num_threads(j);
        full_sum = 0.0;
        start_time = omp_get_wtime();
    }
    ...
```

```
...
#pragma omp parallel private(i)
{
    int id = omp_get_thread_num();
    int numthreads = omp_get_num_threads();
    double x, partial_sum = 0;
#pragma omp single
    for (i = id; i < num_steps; i += numthreads) {
        x = (i + 0.5) * step;
        partial_sum += 4.0 / (1.0 + x * x);
    }
#pragma omp critical
    full_sum += partial_sum;
}
pi = step * full_sum;
run_time = omp_get_wtime() - start_time;
printf("\n pi is %f in %f secs %d threads\n ", pi, run_time, j);
}
}
```

Data flow and Cache Lines

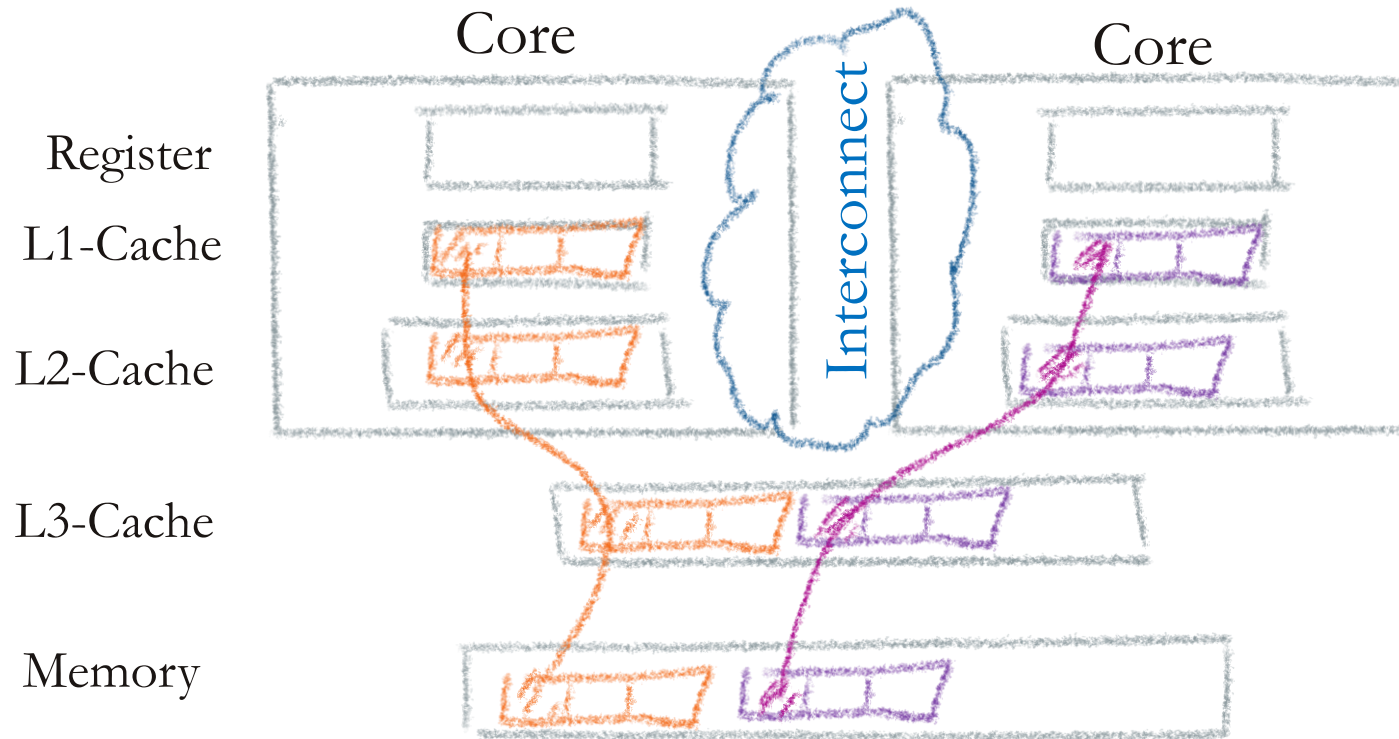


False Sharing



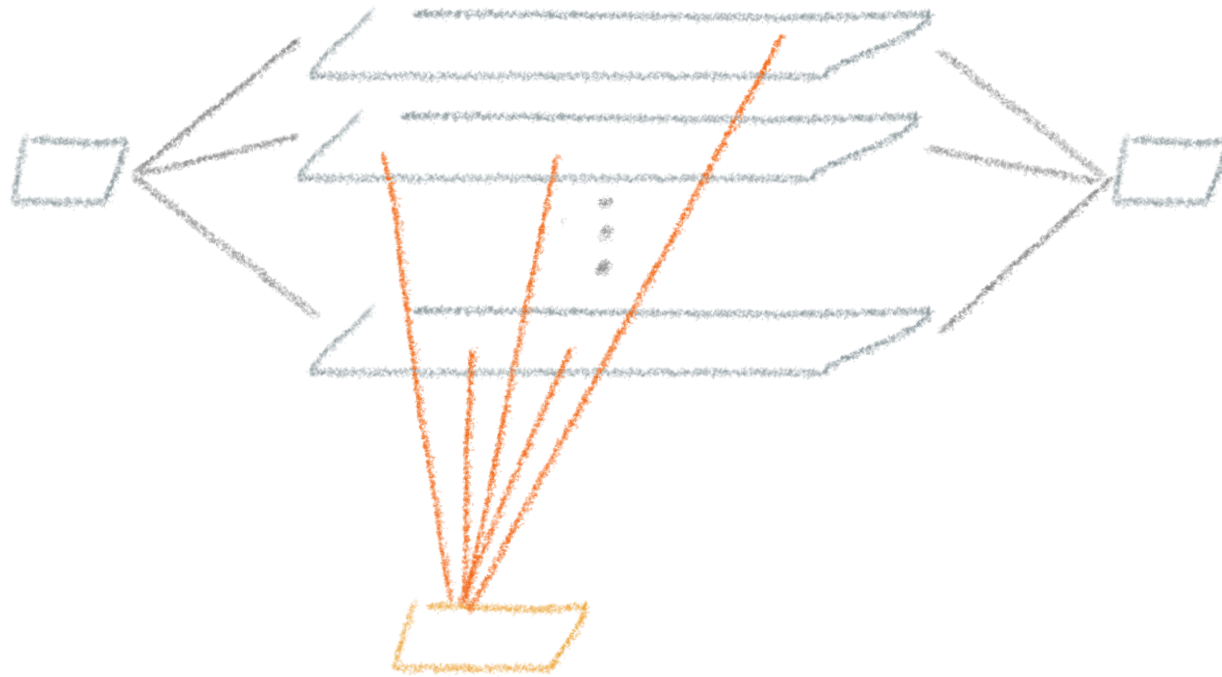
- Different threads use elements of same cache line.
- One of these threads writes to the cache line.
- Threads need to wait for each other.
- Correct computation but performance loss.

Memory alignment



Use a suitable alignment for the data so that two threads do not use the elements of the same cache line to avoid false sharing.

Race Conditions



Memory Location

- Two or more threads access the same memory
- One of these threads writes to a memory location
- If data access is not protected by data sharing attributes or synchronization clauses it might result in unexpected behaviour
- Data race conditions hard to detect in a sandbox/debugger environment

Race Condition Example

```
#include <stdio.h>
#include <omp.h>

void main(){
    int a=0;
    int nt=0;
    #pragma omp parallel num_threads(8)
    {
        int b=a;
        b=b+1;
        a=b;
        nt=omp_get_thread_num();
        printf("Thread_num = %d, a = %d\n", nt, a);
    }
}
```

- Two or more threads access the same memory
- One of these threads writes to a memory location
- If data access is not protected by data sharing attributes or synchronization clauses it might result in unexpected behaviour
- Data race conditions hard to detect in a sandbox/debugger environment

```
➤ gcc -o OpenMP-Synch_Race -fopenmp OpenMP-Synch_Race.c;
   ./OpenMP-Synch_Race
```

```
*****
😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊
*****
```

Synchronization Clauses

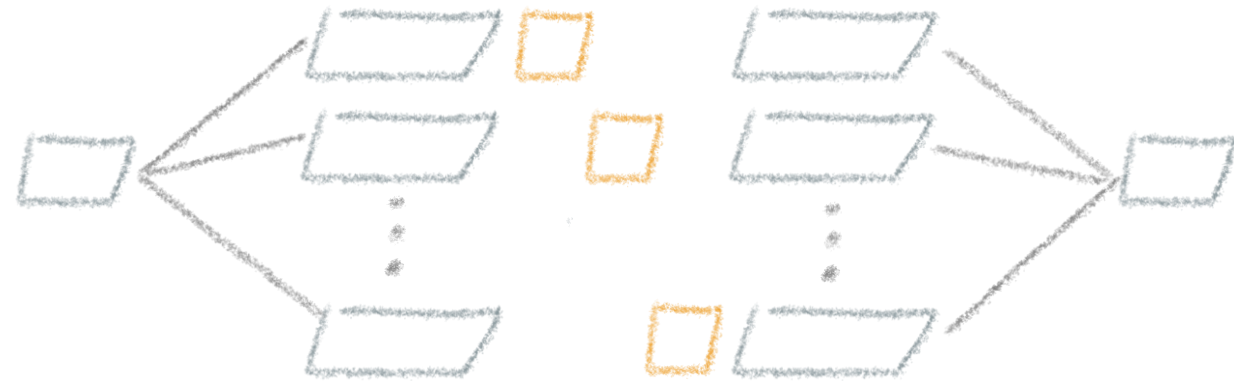
Clause Type	Explanation
critical	Enclosed code block will be executed by only one thread at a time, and not simultaneously executed by multiple threads.
atomic	Memory update (write, or read-modify-write) in the next instruction will be performed atomically. It does not make the entire statement atomic; only the memory update is atomic. A compiler might use special hardware instructions for better performance than when using critical.
barrier	Each thread waits until all of the other threads of a team have reached this point. A work-sharing construct has an implicit barrier synchronization at the end.
nowait	Specifies that threads completing assigned work can proceed without waiting for all threads in the team to finish. In the absence of this clause, threads encounter a barrier synchronization at the end of the work sharing construct.
ordered	Structured block is executed in the order in which iterations would be executed in a sequential loop

Synchronization Critical Example

```
#include <stdio.h>
#include <omp.h>

void main(){
    int a=0;
    int nt=0;
    #pragma omp parallel num_threads(8)
    {
        #pragma omp critical
        {
            int b = a;
            b = b+1;
            a = b;
        }
        nt = omp_get_thread_num();
        printf("Thread_num = %d, a = %d\n", nt, a);
    }
    printf("a = %d\n", a);
}
```

Critical Operation



➤ `gcc -o OpenMP-SyncCritical -fopenmp OpenMP-SyncCritical.c; ./OpenMP-SyncCritical`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Synchronization Atomic

The atomic directive allows access of a specific memory location atomically. It ensures that race conditions are avoided through direct control of concurrent threads that might read or write to or from a particular memory location. With the atomic directive, you can write more efficient concurrent algorithms with fewer locks.

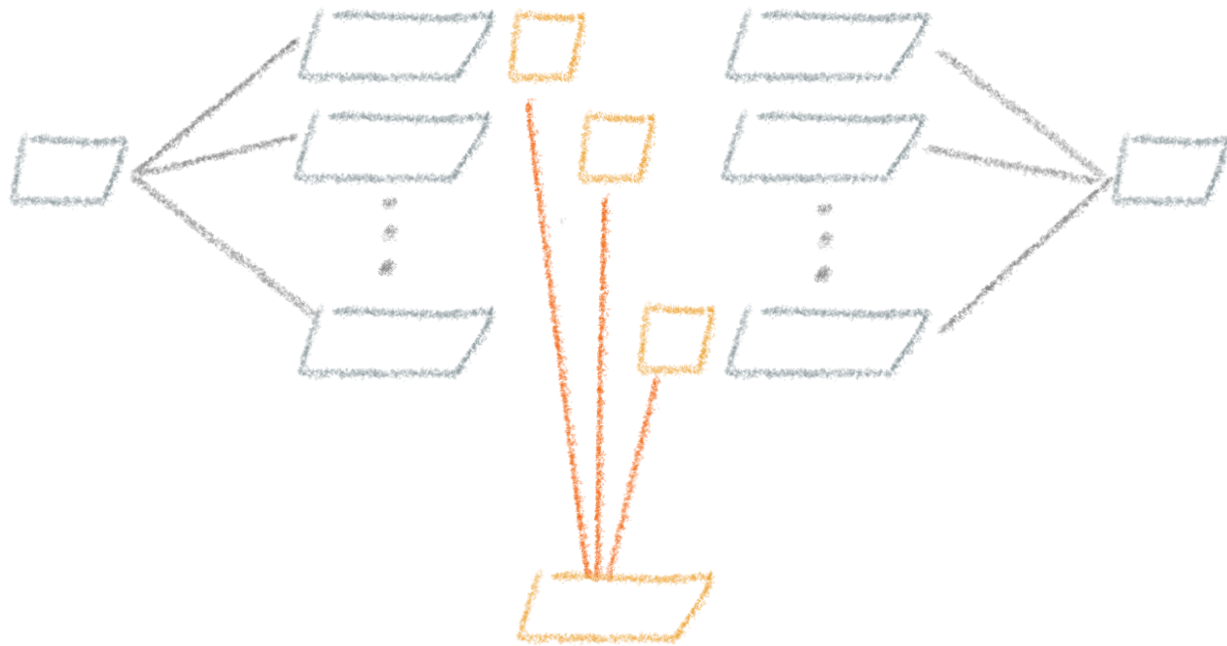
Atomic Clauses	Explanation
update	Update the value of a variable atomically. Only one thread at a time updates the shared variable, avoiding errors from simultaneous writes to the same variable. Atomic directive without a clause is equivalent to atomic update.
read	Reads the value of a variable atomically. The value of a shared variable can be read safely, avoiding the danger of reading an intermediate value of the variable when it is accessed simultaneously by a concurrent thread.
write	Writes the value of a variable atomically. The value of a shared variable can be written exclusively to avoid errors from simultaneous writes.
capture	Updates the value of a variable while capturing the original or final value of the variable atomically.

Atomic Clauses

Directive clause	Expression statement	Structured block
update	<code>x++;</code> <code>x--;</code> <code>++x;</code> <code>--x;</code> <code>x binop = expr;</code> <code>x = x binop expr;</code> <code>x = expr binop x;</code>	
read	<code>v = x;</code>	
write	<code>x = expr;</code>	
capture	<code>v = x++;</code> <code>v = x--;</code> <code>v = ++x;</code> <code>v = --x;</code> <code>v = x binop = expr;</code> <code>v = x = x binop expr;</code> <code>v = x = expr binop x;</code>	<code>{v = x; x binop = expr;}</code> <code>{v = x; xOP;}</code> <code>{v = x; OPx;}</code> <code>{x binop = expr; v = x;}</code> <code>{xOP; v = x;}</code> <code>{OPx; v = x;}</code> <code>{v = x; x = x binop expr;}</code> <code>{x = x binop expr; v = x;}</code> <code>{v = x; x = expr binop x;}</code> <code>{x = expr binop x; v = x;}</code> <code>{v = x; x = expr;}¹ </code>

Synchronization “Atomic” Example

Atomic Operation



Memory Location

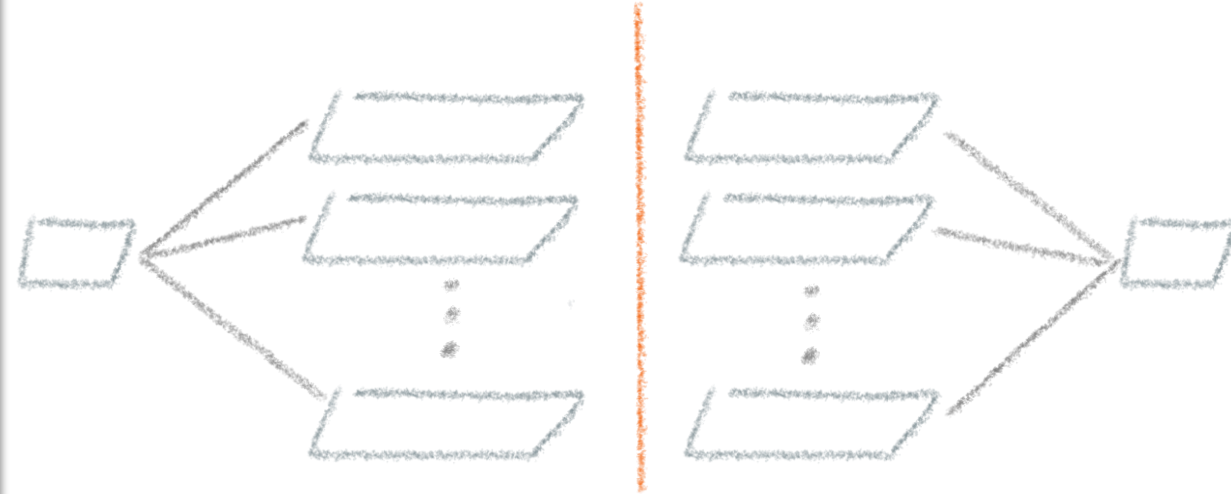
```
#include <stdio.h>
#include <omp.h>
void main(){
    int a=0;
    int nt=0;
    #pragma omp parallel num_threads(8)
    {
        #pragma omp atomic update
        a++;
        nt=omp_get_num_threads();
    }
    printf("Num_threads = %d, a = %d\n", nt, a);
}
```

➤ `gcc -o OpenMP-Synch_Atomic -fopenmp OpenMP-Synch_Atomic.c; ./OpenMP-SynchAtomic`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Synchronization “Barrier” Example

```
#include <omp.h>
#include <stdio.h>
int main(){
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    printf("Hello World...%d", ID);
    #pragma omp barrier
    printf("... from thread = %d\n",ID);
}
return 0;
}
```



Barrier

➤ `gcc -o OpenMP-Synch_Barrier -fopenmp OpenMP-Synch_Barrier.c; ./OpenMP-Synch_Barrier`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Synchronization “NoWait” Example

```
#include <omp.h>
#include <stdio.h>
int main(){
    printf("CP 0\n");
    #pragma omp parallel num_threads(2){
        int ID = omp_get_thread_num();
        printf("CP1 - Thread = %d\n", ID);
        #pragma omp for nowait
        for (int i = 0; i < 3; ++i) {
            printf("CP2 - Thread = %d\n", ID);
        }
        printf("CP3 - Thread = %d\n", ID);
    }
    printf("CP 4\n");
    return 0;
}
```

➤ `gcc -o OpenMP-Synch_NoWait -fopenmp OpenMP-Synch_NoWait.c; ./OpenMP-Synch_NoWait`

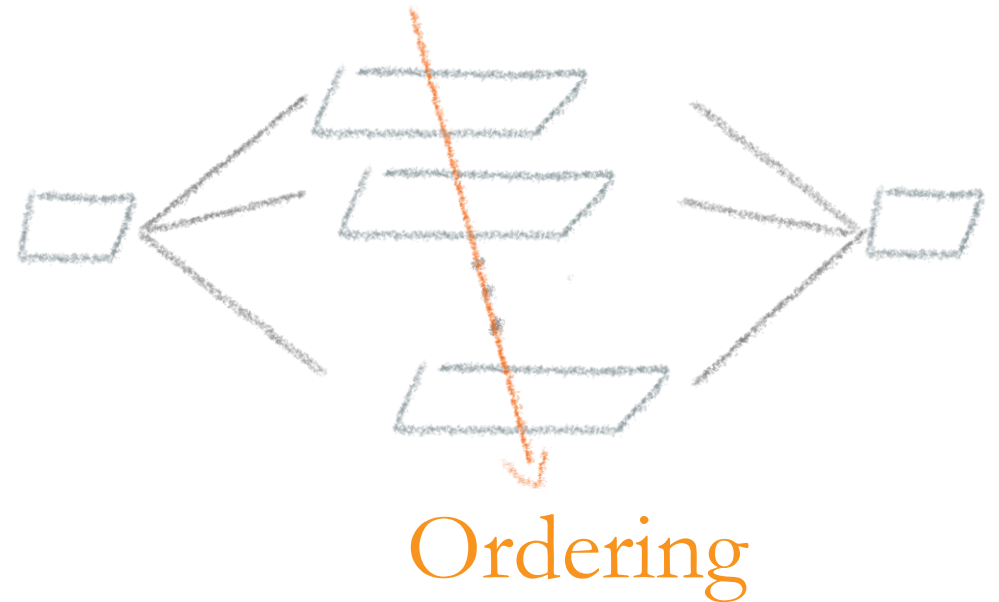
😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊

Synchronization “Ordered” Example

```
#include <stdio.h>
#include <omp.h>
int main(){
    int i=0;
    #pragma omp parallel for ordered
    for(i=0;i<omp_get_num_threads();i++){
        #pragma omp ordered
        printf("NumThread = %d, i= %d\n", omp_get_thread_num(), i);
    }
    return 0;
}
```

➤ `gcc -o OpenMP-Synch_Ordered -fopenmp OpenMP-SynchOrdered.c; ./OpenMP-Synch_Ordered`

😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊



Scheduling Clauses

Schedule clause	Explanation
<code>schedule (type, chunk)</code>	The iterations in the work sharing construct (given by a do-loop or for-loop) are assigned to threads according to the scheduling method defined by this clause

Schedule Types	Explanation
static	All the threads are allocated iterations before they execute the loop iterations. The iterations are divided among threads equally by default. However, specifying an integer for the parameter chunk will allocate chunk number of contiguous iterations to a particular thread.
dynamic	Some of the iterations are allocated to a smaller number of threads. Once a particular thread finishes its allocated iteration, it returns to get another one from the iterations that are left. The parameter chunk defines the number of contiguous iterations that are allocated to a thread at a time.
guided	A large chunk of contiguous iterations are allocated to each thread dynamically (as above). The chunk size decreases exponentially with each successive allocation to a minimum size specified in the parameter chunk.
...	